1. Gramática y símbolos de entrada

Unidad de análisis: `int main() Block`. Se conservan las producciones 0–50 y la numeración de los 161 estados CLR(1).

Terminales y normalización

La entrada está formada por tokens del lexer anterior. Cada token conserva `type`, `lexeme`, `line` y `column`. La etiqueta gramatical normalizada es `terminal`. El parser trabaja con tokens, no con caracteres sueltos.

Entrada del lexer	Terminal del parser
Palabra clave admitida	Su lexema: `int, float, char, if, else, while, return`.
Identificador	`ID`; el lexema `main` se normaliza como `main`.
Operador o puntuación	Su lexema completo: `==` no se forma uniendo dos tokens `=`.
Constante válida	`INT, FLOAT, CHAR, STRING`, según su clase léxica.
Fin de entrada	`$`, una sola vez. Un error léxico bloquea el parser.

Los 31 terminales del lenguaje son: `! != % ( ) * + - / ; < <= = == > >= CHAR FLOAT ID INT STRING char else float if int main return while { }`. `int` es una palabra clave; `INT`, una constante. `epsilon` no es token. ACTION añade `$` como columna 32.

No terminales y símbolo inicial

`Program, Block, Items, Item, Decl, Type, Init, Stmt, Matched, Unmatched, Simple, Expr, Equality, Relation, Add, Mul, Unary, Primary`. El símbolo inicial es `Program`. La aumentación agrega `Start -> Program` (p0). Las listas vacías y los inicializadores ausentes tienen una parte derecha vacía.

Unidad de entrada y declaraciones

La unidad completa reconocida por Program contiene una función `int main()` y su bloque. Por ejemplo:

```
int main() {
    int x = 10;
    return x;
}
```

La declaración `int x = 10;` es válida dentro del bloque, pero no constituye un Program completo por sí sola. Las declaraciones y las asignaciones se distinguen por su estructura: la declaración comienza por Type y la asignación por ID.

Producciones completas

La numeración es la usada por las reducciones y las acciones SDT.

ID	Producción	Función
0	`Start -> Program`	Regla aumentada; aceptación.
1	`Program -> int main ( ) Block`	Unidad de entrada.
2	`Block -> { Items }`	Bloque y ámbito.
3	`Items -> Items Item`	Agrega un elemento.
4	`Items -> epsilon`	Lista vacía.
5	`Item -> Decl`	Elemento de declaración.
6	`Item -> Stmt`	Elemento de sentencia.
7	`Decl -> Type ID Init ;`	Variable e inicializador opcional.
8	`Type -> int`	Tipo entero.
9	`Type -> float`	Tipo real.
10	`Type -> char`	Tipo carácter.
11	`Init -> = Expr`	Inicializador presente.
12	`Init -> epsilon`	Sin inicializador.
13	`Stmt -> Matched`	Sentencia cerrada.
14	`Stmt -> Unmatched`	Sentencia abierta.
15	`Matched -> Simple`	Instrucción simple.
16	`Matched -> Block`	Bloque delimitado.
17	`Matched -> if ( Expr ) Matched else Matched`	if/else cerrado.
18	`Matched -> while ( Expr ) Matched`	while de cuerpo cerrado.
19	`Unmatched -> if ( Expr ) Stmt`	if pendiente de else.
20	`Unmatched -> if ( Expr ) Matched else Unmatched`	if/else de cierre pendiente.
21	`Unmatched -> while ( Expr ) Unmatched`	while de cuerpo abierto.
22	`Simple -> ID = Expr ;`	Asignación.
23	`Simple -> return Expr ;`	Retorno con expresión.
24	`Simple -> ;`	Sentencia vacía.
25	`Expr -> Equality`	Entrada a expresión.
26	`Equality -> Equality == Relation`	Igualdad.
27	`Equality -> Equality != Relation`	Desigualdad.
28	`Equality -> Relation`	Paso a relaciones.
29	`Relation -> Relation < Add`	Menor.
30	`Relation -> Relation <= Add`	Menor o igual.
31	`Relation -> Relation > Add`	Mayor.
32	`Relation -> Relation >= Add`	Mayor o igual.
33	`Relation -> Add`	Paso a suma.
34	`Add -> Add + Mul`	Suma.
35	`Add -> Add - Mul`	Resta.
36	`Add -> Mul`	Paso a producto.
37	`Mul -> Mul * Unary`	Multiplicación.
38	`Mul -> Mul / Unary`	División.
39	`Mul -> Mul % Unary`	Módulo entero.
40	`Mul -> Unary`	Paso a unario.
41	`Unary -> + Unary`	Positivo unario.
42	`Unary -> - Unary`	Negativo unario.
43	`Unary -> ! Unary`	Negación lógica.
44	`Unary -> Primary`	Paso a primario.
45	`Primary -> ID`	Uso de identificador.
46	`Primary -> INT`	Constante entera.
47	`Primary -> FLOAT`	Constante real.
48	`Primary -> CHAR`	Literal carácter.
49	`Primary -> STRING`	Literal cadena.
50	`Primary -> ( Expr )`	Agrupación explícita.

Precedencia y asociatividad

De mayor a menor prioridad: primarios y paréntesis; prefijos `+ - !`; `* / %`; `+ −`; `< <= > >=`; `== !=`. Los prefijos se asocian a la derecha y los binarios a la izquierda. La asignación solo es sentencia o inicialización, no parte de Expr.

`Matched/Unmatched` asocia cada `else` con el `if` abierto más cercano. Las variantes de `while` conservan esa distinción. Las declaraciones como cuerpo de control requieren llaves.

Cómo reconocer una declaración

Para `int x = 10;` dentro de main, el lexer entrega `int ID = INT ;`. Los lexemas `x` y `10` permanecen asociados a sus tokens. La gramática organiza esa secuencia con estas reglas:

```
p7:  Decl -> Type ID Init ;
p8:  Type -> int
p11: Init -> = Expr
p46: Primary -> INT
```

Entre Expr y Primary se aplican las producciones de paso 25, 28, 33, 36, 40 y 44. No se omiten al construir el árbol. Así, la declaración tiene cuatro hijos: Type, ID, Init y el punto y coma.

En `int x;` se usa p12, `Init -> epsilon`. Init sigue siendo un nodo del árbol, pero no tiene hijos. La declaración no cambia de producción: sigue usando p7.

Función de los grupos de no terminales

Grupo	Estructura que representa
Program y Block	La unidad completa y un cuerpo delimitado por llaves.
Items e Item	Una lista de declaraciones y sentencias; Items también permite una lista vacía.
Decl, Type e Init	Nombre, tipo e inicialización opcional de una variable.
Stmt, Matched y Unmatched	Sentencias y asociación no ambigua de if/else.
Simple	Asignación, retorno o punto y coma aislado.
Expr hasta Primary	Niveles de prioridad de las expresiones.

Ejemplos de prioridad y agrupación

`2 + 3 * 4` se reconoce como `2 + (3 * 4)` porque Add utiliza un Mul completo como operando. `8 - 3 - 1` se agrupa como `(8 - 3) - 1` por la recursión izquierda de Add. Los paréntesis de `(2 + 3) * 4` fuerzan primero la suma.

La forma `if (x) if (y) ; else ;` vincula el else con el if de y. Matched representa la sentencia interior con else; Unmatched permite que el if exterior siga abierto. No se resuelve este caso dando prioridad manual al token else en ACTION.

Límites y verificación

Se admiten bloques, declaraciones de una variable, asignaciones, expresiones, `if/else`, `while` y `return Expr;`. Se excluyen otras funciones y llamadas, declaraciones globales, arreglos, apuntadores, `for`, `switch`, `struct`, preprocesador y operadores no enumerados. `STRING` permite reconocer sintácticamente `int x="hello";` y rechazarlo después en SDT; no añade un tipo declarable de cadena.

La gramática produce 161 estados canónicos y 730 transiciones. Su tabla ACTION/GOTO no presenta conflictos shift/reduce ni reduce/reduce.